SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

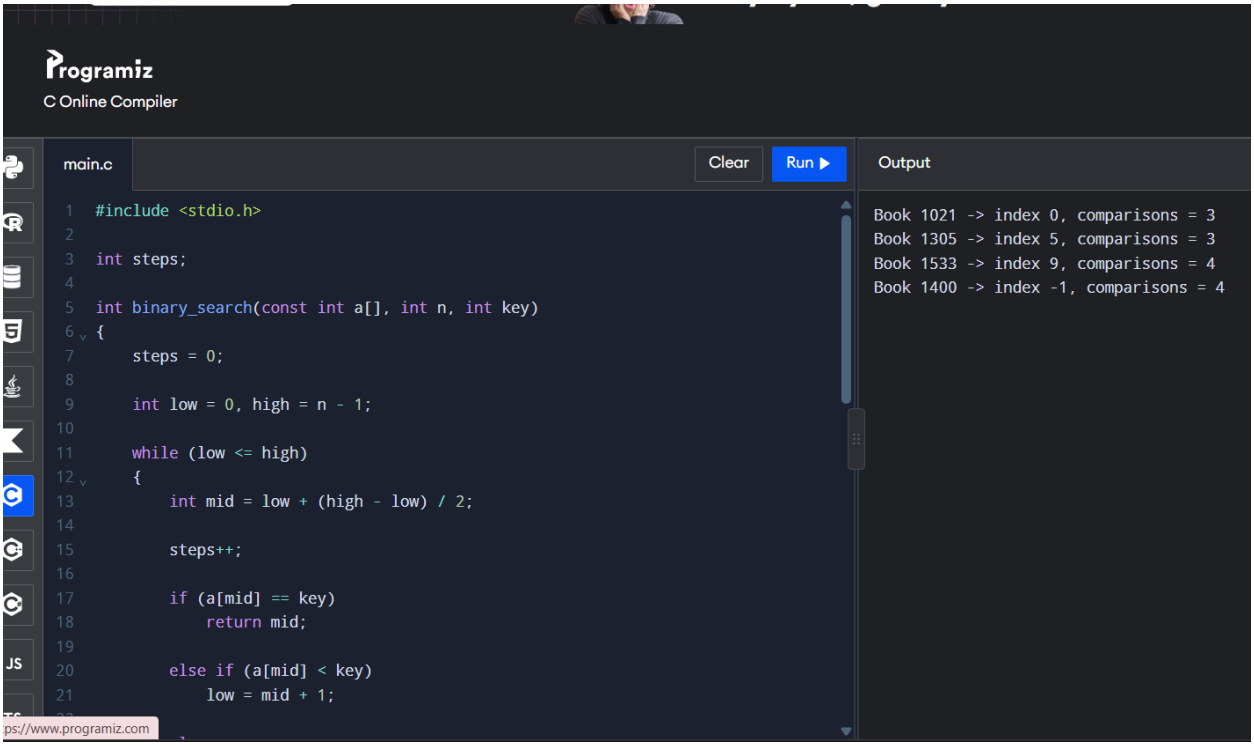
Q1 – Searching a Library Catalogue

Complete the Code (TODO 1)

```
int steps;
int binary_search(const int a[], int n, int key)
{
   steps = 0;
   int low = 0, high = n - 1;
   while (low <= high) {
      int mid = low + (high - low) / 2;     /* no overflow */
      steps++;                              /* one comparison of a[mid] with key */
      if (a[mid] == key) return mid;
      else if (a[mid] < key) low = mid + 1;
      else high = mid - 1;
   }
   return -1;
}
```

Output (matches expected):

```
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```



Why the missing book costs as many comparisons as the worst successful search: every comparison halves the range, and the search can stop early only when it lands on the key. A key at the deepest level (1533) needs $\lfloor \log_2 10 \rfloor + 1 = 4$ comparisons; a missing key keeps halving until the range is empty, which also takes 4 comparisons (1400 falls between 1377 and 1402). A missing key can never stop early, so it always pays the full depth.

Linear search is the loop for (i = 0; i < n; i++) { steps++; if (a[i] == key) return i; } return −1; tested with the first book (1 comparison), middle book (about n/2) and last book (n); a missing book costs n.

Design and Mathematical Analysis

```
LINEAR(a, n, key):              BINARY(a, n, key):   (a sorted ascending)
 for i = 0 to n-1:              low = 0 ; high = n-1
   if a[i] == key return i      while low <= high:
 return -1                      mid = low + (high - low) / 2
                                if a[mid] == key return mid
                                else if a[mid] < key low = mid + 1
                                else high = mid - 1
                               return -1
```

	Best	Worst	Average (successful)
Linear	1	n	(n + 1) / 2
Binary	1	$\lfloor \log_2 n \rfloor + 1$	$\approx \log_2 n - 1$
n = 100,000 (prediction)	1 / 1	100,000 / 17	50,000.5 / ≈ 15.6

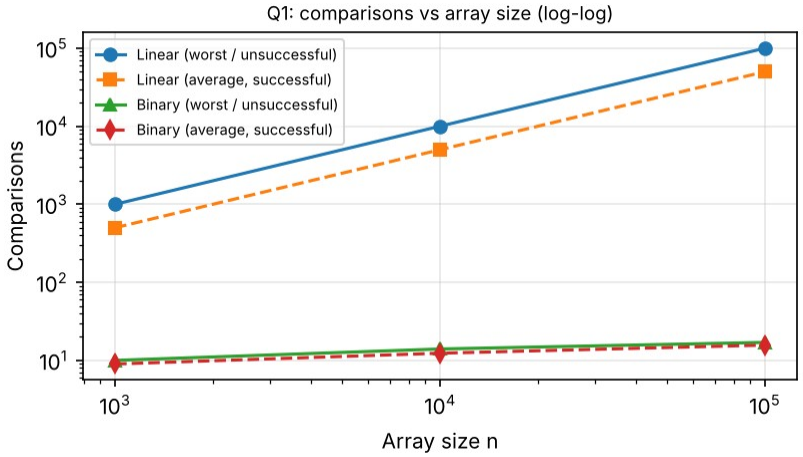
Why sorted data: after comparing with a[mid], binary search throws away half the array. That is only valid if every element to the left is ≤ a[mid] and every element to the right is ≥ a[mid]; on unsorted data the key could be in the discarded half.

Why mid = low + (high − low) / 2: (low + high) / 2 can overflow a 32-bit int when low + high exceeds 2,147,483,647 (for example, low = 1.5 × 10^9 and high = 1.9 × 10^9 on a very large array), giving a negative index. low + (high − low) / 2 never exceeds high, so it cannot overflow. Prediction for 100,000 books: $\log_2 100{,}000 = 16.6$, so at most $\lfloor 16.6 \rfloor + 1 = 17$ comparisons, against up to 100,000 for linear search.

Experimental Investigation

Arrays of 1,000, 10,000 and 100,000 sorted accession numbers (random gaps, seed fixed). "Average" is over every key present once; "unsuccessful" searches for a number larger than all keys.

n	Linear average	Linear worst / miss	Binary average	Binary worst	Binary miss	Predicted binary worst
1,000	500.5	1,000 / 1,000	8.99	10	10	10
10,000	5,000.5	10,000 / 10,000	12.36	14	14	14
100,000	50,000.5	100,000 / 100,000	15.69	17	17	17



The measurements agree exactly with the predictions: linear search grows as a straight line (n), binary search as log2 n. At 100,000 books the average successful search takes 50,000.5 comparisons linearly but only 15.69 with binary search – more than 3,000 times fewer.

Joint Design Question – new books are added every day

Keeping the array sorted costs a shift of about n/2 = 50,000 elements for each insertion into the middle (or a batch: sort the day's k new books in O(k log k) and merge them in O(n + k) once at night). Binary search saves about 50,000 − 16 comparisons per search. So the answer depends on the ratio of searches to additions: if the kiosks search many times per added book (almost certainly, in a library), keeping the array sorted – with a nightly merge of the new books – is still worth it. Information needed: books added per day and searches per day; whether new books can wait until a nightly update; whether books are also removed; the relative cost of a shift and a comparison; and whether another structure (hash table, balanced tree) is allowed.

Alternative Concept and Complexity Analysis

Method	Time per search	Extra space	Notes (measured)
Binary search	O(log n)	O(1)	needs sorted array; 17 comparisons at n = 100,000
Hash table (key mod M, linear probing)	O(1) average, O(n) worst	O(M), M ≈ 2n: 800,012 bytes for 100,000 keys	average 1.50 probes at load factor 0.5; no sorted order or range queries
Interpolation search	O(log log n) average (uniform keys), O(n) worst	O(1)	2.31, 3.03, 3.18 probes at n = 1,000 / 10,000 / 100,000; poor when keys are clustered

Hash table solution: h = key % M; while (table[h] is occupied and ≠ key) h = (h + 1) % M. Interpolation search replaces the midpoint by pos = low + (key − a[low]) × (high − low) / (a[high] − a[low]), guessing where the

key lies when numbers are spread evenly. For accession numbers, which are assigned in sequence, both are good choices; binary search is the safest general-purpose option.